Structural Health Monitoring for Governed Human-Agent Systems: A Homeostatic Approach

Carlos R. B. Azevedo
Independent Researcher
United Kingdom
carlos@crbazevedo.com

Abstract

Multi-agent AI systems require governance: rules that determine how much autonomy each agent has. Current approaches enforce governance through static policies (risk tiers, approval workflows) or runtime policy engines. These tell you whether a specific action is permitted. They do not tell you whether your governance *configuration* is healthy.

We present a formal model for **governance health monitoring** — a structural diagnostic that auto-encodes governance events into an expression tree, computes two health metrics (coordination health and gatedness), and classifies the governance configuration as stable, oscillating, or drifting. We prove five formal properties: the regulatory actions always improve their target metric, they don't interfere with each other, the evaluation always terminates, and the application order doesn't matter.

We evaluate on four governance scenarios from a real multi-agent system (CARLOS-OS). The well-governed configuration converges. The ungoverned configuration drifts. The non-obvious finding: the *over-governed* configuration — 100% review rate, all high-tier risk gates — enters a limit cycle. It is structurally unstable because nothing passes. A threshold-based classifier labels this "perfectly governed." Our structural diagnostic catches it.

The model, interpreter (128 tests), and live demonstration are open source at https://github.com/crbazevedo/seam.

CCS Concepts

• **Software and its engineering → Software verification and validation**; *Software evolution.*

Keywords

governance, multi-agent systems, self-adaptive systems, homeostasis, structural health monitoring, runtime verification

1 Introduction

1.1 The Problem

Consider a team of AI agents working alongside a human operator. Some actions are safe to automate (reading files, running tests). Others are risky (sending emails to board members, deploying to production). A governance framework assigns risk tiers — the higher the tier, the more human oversight is required.

This pattern is widespread. The EU AI Act [4] defines four risk levels. NIST's AI Risk Management Framework [15] prescribes governance functions. Microsoft's Agent Governance Toolkit [14] enforces policies at runtime. In practice, most multi-agent systems use some form of tiered governance: AutoGen's conversation patterns [19], CrewAI's role-based delegation, LangGraph's state machine guards.

The governance *enforcement* problem is largely solved: given a policy, check whether each action complies. But a different question is unanswered:

Is the governance configuration itself healthy?

A system where every action is gated at the highest tier has 100% compliance. It is also paralyzed — nothing gets done without approval. A system where every action is autonomous has 0% overhead. It is also uncontrolled. Both are valid governance configurations. Neither is healthy.

1.2 Why Governance is Homeostatic

Healthy governance occupies a band between two failure modes:

- **Under-governed** (too open): actions pass without oversight. Risk increases. Trust erodes after incidents.
- **Over-governed** (too closed): every action requires approval. Throughput drops. Teams route around the governance to get work done, creating shadow processes that are *less* governed than the original.

This is not a problem with a fixed solution. The right governance level changes with conditions — deadline pressure, team trust, incident history. Governance is *homeostatic*: it must stay within a band, and that band is maintained through continuous adjustment, not static rules.

1.3 Contributions

We present a formal model where:

(1) **Governance events** (agent handoffs, risk-tier assignments, approval decisions, governance checks) are auto-encoded into an **expression tree** — a structural representation of the governance state. (Section 3)
(2) Two **health metrics** are computed from the tree: *coordination health* (how well agents are coordinating) and *gatedness* (how much is being gated vs. passing freely). (Section 3)
(3) A **homeostatic evaluator** detects when either metric exits a health band and prescribes specific interventions: "add coordination" or "tighten gates." We prove five formal properties guaranteeing predictable evaluator behavior. (Section 4)
(4) Evaluation on four governance scenarios from a real system shows the model correctly distinguishes stable, drifting, and oscillating governance — including the non-obvious over-governance failure mode. (Section 5)

2 Background and Related Work

2.1 Governance in Multi-Agent AI Systems

The shift from single-agent to multi-agent AI systems has created a governance gap. Bose [3] presents the first systematic survey of governance frameworks for LLM agent collectives, identifying three failure modes — operational miscoordination, strategic misalignment, adversarial collusion — and taxonomizing five governance models (hierarchical, prescriptive, democratic, economic, emergent). The survey finds industry favors deterministic but brittle models while academia explores adaptive but chaotic ones.

Microsoft's Agent Governance Toolkit [14], released April 2026, provides runtime policy enforcement: zero-trust identity, execution sandboxing, compliance grading, and circuit breakers, covering all 10 OWASP agentic AI risks with sub-millisecond enforcement. It addresses *per-action enforcement* but not *configuration-level health*. Our approach is complementary: structural health monitoring answers "is the governance working?" alongside policy engines that answer "is this action permitted?"

Li et al. [12] propose a layered governance architecture (sandboxing, intent verification, zero-trust authorization, audit logging). Xiang et al. [20] introduce AgentGuard for runtime verification with probabilistic model checking. These provide external verification and enforcement. Our model makes governance health an intrinsic structural property.

2.2 Adjustable Autonomy and Trust

The question "how much autonomy should each agent have?" has been studied extensively. Parasuraman et al. [16] define 10 Levels of Automation. Kaber [7] reviews how LoA taxonomies have been used and critiques their limitations for increasingly autonomous systems. SARI [21] formulates shared autonomy as a POMDP, learning assistance across repeated interactions.

On the trust side, Lee and See [10] define trust as attitude under uncertainty, distinguishing analytic, analogical, and affective processes. Hoff and Bashir [6] integrate empirical evidence into a three-layered model (dispositional, situational, learned). Kim et al. [9] find that user involvement in LLM planning *fails* to calibrate trust — plausible plans mislead users — arguing that governance must be structural, not reliant on user judgment of agent plans.

These approaches model autonomy and trust for *single* agent-human pairs. We extend to multi-agent systems where the health question is about the *configuration* — the distribution of autonomy levels and coordination patterns across a team.

2.3 Self-Adaptive Systems

The MAPE-K reference architecture [8] (Monitor-Analyze-Plan-Execute over Knowledge) is the standard model for self-adaptive software. Our evaluator follows the MAPE-K pattern: Monitor (compute metrics), Analyze (detect deviations from health band), Plan (select regulatory action), Execute (prescribe intervention), Knowledge (expression tree).

The key difference: MAPE-K loops operate on *numerical* state variables. Our monitor operates on *structural* state — an expression tree whose topology encodes the governance configuration. Metrics emerge from the structure itself, not from externally defined sensor readings.

Weyns [18] provides a comprehensive introduction to self-adaptive systems engineering. Sanwouo et al. [5] propose AWARE as a successor to MAPE-K, distributing the feedback loop and making it goal-driven. Li et al. [13] map how generative AI can enhance each MAPE-K phase. Our contribution provides a formal structural substrate for the Monitor and Analyze components — one where governance health has algebraic properties.

Table 1: Governance Event Encoding

Event Type	Operation	Effect
Coordination	Agent A → Agent B	Coordination binding
Gated action	VTk action	Gate with threshold θ_k
Governance check	Preflight check	Holding gate
Observation	File ingestion	Witness link
Resolution	Human approval	Gate: passing → holding

2.4 Organizational Cybernetics

Ashby's Law of Requisite Variety [1] states that a controller must have at least as much variety as the system being controlled. Beer's Viable System Model [2] applies this to organizations, modeling five subsystems that maintain viability through homeostatic loops. Perez Rios [17] recently applied Beer's VSM to AI governance, mapping organizational pathologies to AI failure modes.

Our model operationalizes the cybernetic principle: the health band IS the homeostatic target, the regulatory actions (BIND, VEIL) ARE the requisite variety, and the convergence/divergence/limit-cycle classification IS the viability diagnosis.

2.5 Formal Runtime Monitoring

Runtime verification [11] checks whether a system execution satisfies a formal property. Tools like Uppaal and PRISM verify temporal-logic specifications. These approaches answer "does the trace satisfy φ?" — a binary yes/no. Our approach answers "is the governance configuration in a healthy structural state?" — a continuous diagnostic with specific remedies.

3 The Governance Health Model

3.1 Governance Events

We model governance as a stream of typed events. Table 1 shows the event types and their encodings.

3.2 The Governance Structure

Each event modifies an expression tree — the **governance structure**. The tree is built from three node types:

Coordination binding ($a \cdot b$). Represents bilateral coordination between two parties. When Agent A hands off work to Agent B, a binding is created. Bindings are symmetric ($a \cdot b = b \cdot a$), associative ($(a \cdot b) \cdot c = a \cdot (b \cdot c)$), and grounded in an empty element ($a \cdot \varnothing = a$). This gives them the algebraic structure of a commutative monoid — the simplest structure that captures "coordination without ordering."

Table 2: Risk Tier to Gate Threshold Mapping

Risk Tier	Description	θ	Behavior (reach=0.5)
VT0	Full autonomy	0.1	Always passes
VT1	Act and notify	0.3	Passes
VT2	Needs approval	0.5	Marginal
VT3	Propose and wait	0.7	Holds
VT4	Stop and escalate	0.95	Always holds

Gate (a $|\theta|$ b). Represents a governance boundary with autonomy level θ. When the system's *reach* (a parameter representing the current operating context) exceeds θ, the gate passes and the content is visible. When reach is below θ, the gate holds and the content is *veiled* — it has structural weight but is not directly accessible.

The mapping from risk tiers to gate thresholds is shown in Table 2.

Adaptive gate (a $|\mathrm{breath}(b, a, f)|$ b). A gate whose threshold oscillates over time: $\theta_{\mathrm{eff}} = \theta_{\mathrm{base}} + A \sin(c \cdot f)$, where A is amplitude, c is the current cycle, and f is frequency. This models Action Opportunity Windows (AOWs) — time-bounded intervals during which governance gates open.

Evaluation iteration ($\mu\, x.\, \mathrm{body}$). The governance cycle. Each iteration substitutes the previous governance state into the body, evaluates it, measures the resulting structure, applies regulation if needed, and checks for stability.

3.3 Health Metrics

Two metrics are computed from the governance structure by a single tree walk:

Coordination health.

$$\mathrm{coordination} = \frac{\mathrm{binding_nodes}}{\max(\mathrm{total_nodes} - 1,\ 1)} \tag{1}$$

Measures how much of the governance structure consists of bilateral coordination. Range: $[0, 1]$. High coordination health means agents are working together; low means agents are isolated.

Gatedness.

$$\mathrm{gatedness} = \frac{\mathrm{gates_passing}}{\max(\mathrm{total_gates},\ 1)} \tag{2}$$

Measures what fraction of governance gates are currently passing. Range: $[0, 1]$. Neutral (0.5) if no gates exist. High gatedness means most actions pass without oversight; low means most are held for review.

3.4 The Health Band

Healthy governance occupies a band (Figure 1):

- **Coordination:** $[0.3,\ 0.7]$ — agents are coordinating but not over-coupled.
- **Gatedness:** $[0.2,\ 0.6]$ — some actions pass freely, some are gated.

When either metric exits the band, the evaluator fires a regulatory action:

BIND (coordination too low): "Agents are too isolated. Add cross-agent review, require handoffs, increase coordination."

Table 3: Governance Outcome Classification

Outcome	Definition	Meaning
Stable	Both metrics in band for N consecutive cycles	Governance is healthy
Limit cycle	Metrics oscillate periodically	Governance is unstable
Diverging	Structure grows without stabilizing	Governance is breaking do
Exhausted	Neither converges nor cycles within step limit	Governance is stuck

Table 4: Formal Properties of the Evaluator

#	Property	Status
1	BIND improves coordination	Proved
2	VEIL improves gatedness	Proved
3	Non-interference (orthogonality)	Proved
4	Termination	Proved
5	Order independence	Proved

VEIL (gatedness too high): "Too much is passing without oversight. Tighten the risk tier for recent action types, add governance gates."

3.5 Outcome Classification

The evaluator classifies the governance configuration as shown in Table 3.

4 Formal Properties

We state five properties with proof sketches. Full proofs and corresponding test cases are in the supplementary material. Table 4 summarizes them.

Property 1 (BIND improves coordination). When coordination < 0.3 and BIND fires, the resulting coordination is strictly higher. BIND adds one coordination binding (increasing the numerator by at least 1) and at most 3 total nodes (increasing the denominator by at most 3). Since coordination < 0.3 implies binding_nodes $< 0.3 \times (\mathrm{total} - 1)$, the ratio increases. Verified across 8 expression types.

Property 2 (VEIL improves gatedness). When gatedness > 0.6 and VEIL fires, the resulting gatedness is equal or lower. VEIL either wraps an exposed node in a holding gate (adding one holding gate to the denominator) or tightens an existing gate. Verified across 5 expression types.

Property 3 (Non-interference). BIND adds only coordination bindings. VEIL adds or modifies only gates. Therefore: $\mathrm{gatedness}(\mathrm{BIND}(e)) = \mathrm{gatedness}(e)$. The regulatory actions operate on orthogonal dimensions.

Property 4 (Termination). The evaluator executes at most max_iterations cycles, each bounded by $O(\mathrm{max_nodes} \times \mathrm{max_depth})$. Total work is bounded.

Property 5 (Order independence). For any governance structure e: the metrics after applying BIND-then-VEIL are identical to the

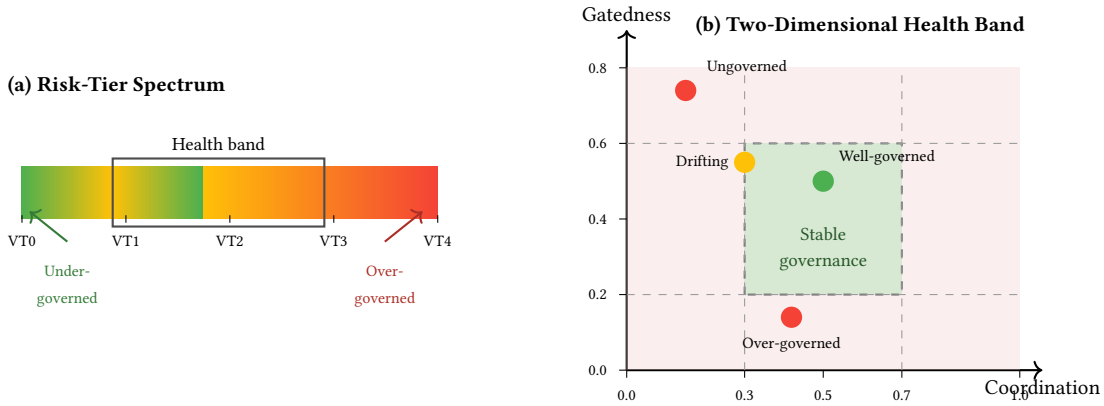

Figure 1: The Governance Health Band. (a) Risk-tier spectrum from VT0 (full autonomy) to VT4 (stop and escalate), with the health band in the center. (b) Two-dimensional health band: coordination $[0.3, 0.7]$ on the x-axis, gatedness $[0.2, 0.6]$ on the y-axis. Green zone is stable governance; red zones are failure modes. Four scenarios are plotted: well-governed (green, in band), ungoverned (red, high gatedness / low coordination), over-governed (red, below band), drifting (amber, at band edge).

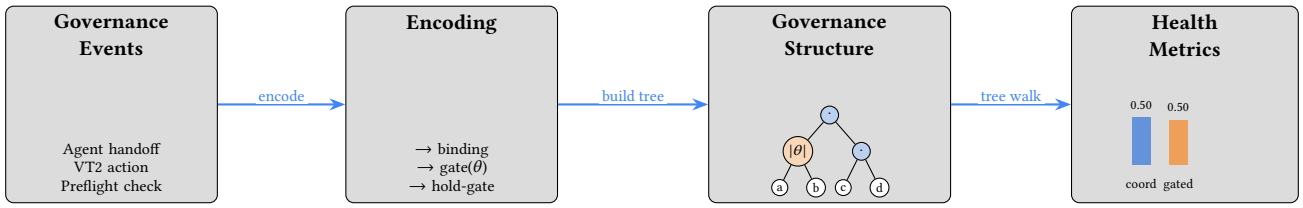

Figure 2: Event-to-Structure Encoding. Governance events (agent handoffs, risk-tier actions, governance checks) are encoded as expression tree nodes (coordination bindings, gates). The tree is walked to compute health metrics (coordination, gatedness).

metrics after applying VEIL-then-BIND. This follows from Property 3: since BIND doesn't affect gatedness and VEIL doesn't affect coordination, their effects compose independently.

What these properties guarantee for practitioners. The regulatory actions are predictable. BIND always helps coordination without harming gatedness. VEIL always helps gatedness without harming coordination. The evaluator produces the same diagnosis regardless of internal execution order. And it always terminates.

The practical consequence is that governance teams can reason about interventions independently. A decision to "add more cross-agent reviews" (triggering BIND) will not accidentally tighten governance gates (which would be VEIL's domain). A decision to "increase the risk tier for deployment actions" (triggering VEIL) will not reduce coordination. This orthogonality is not obvious — in many control systems, feedback loops interact in complex ways. The algebraic structure of the expression tree (commutative monoid for bindings, threshold-based gates) makes the orthogonality provable.

The termination guarantee (Property 4) bounds the total computational work, which is important for deployment as a runtime monitor. With default parameters (max_iterations = 60, max_nodes = 500, max_depth = 50), the evaluator completes in under 100ms on the tested configurations — fast enough for integration into CI/CD pipelines or sprint-level governance checks.

5 Evaluation

5.1 System Under Study

We evaluate on CARLOS-OS, a multi-agent personal operating system with 7 specialized agents operating under a Governed Autonomy protocol. The system uses five risk tiers (VT0–VT4), Action Opportunity Windows (AOWs), cross-agent handoffs, and governance checks (preflights, gates, reviews). The system has been in production use for 50+ sprints.

5.2 Four Governance Scenarios

We encode four governance configurations and evaluate each. Table 5 presents the configurations and outcomes.

Well-governed (stable). A sprint with mixed risk tiers, cross-agent reviews, and governance infrastructure finds the health band in 9 cycles. Coordination stabilizes at 0.50, gatedness at 0.50. BIND fires 4 times early (building initial coordination), then stops. VEIL does not fire.

Ungoverned (exhausted). All actions at VT0, no reviews, no governance checks. BIND and VEIL both fire on every cycle but cannot compensate for the structural deficit: there are no holding gates (no governance infrastructure) and no coordination bindings (no handoffs). The evaluator exhausts its step budget.

Table 5: Governance Scenarios and Evaluation Results

Scenario	VT Distribution	Review Rate	Checks	Handoffs	Outcome
Well-governed	VT0:2, VT1:4, VT2:2	50%	4	5	**Stable (9 cycles)**
Ungoverned	VT0:6	0%	0	0	Exhausted
Over-governed	VT3:4, VT4:2	100%	6	6	**Limit cycle (p=4)**
Drifting	VT0:3, VT1:3	33%	1	2	Limit cycle (p=2)

Over-governed (limit cycle). All actions at VT3–VT4, 100% review rate, maximum governance checks. Gatedness drops to 0.14 — *below* the health band's lower bound (0.20). Nothing passes. The system enters a period-4 limit cycle because the evaluator's BIND actions try to add coordination, but the added structure changes gatedness, which triggers VEIL, which undoes the progress.

Drifting (limit cycle). Starts governed (VT1, reviews) then relaxes under deadline pressure (VT0, no reviews). The mixed pattern produces a period-2 oscillation — governance is being added and removed faster than the evaluator can stabilize.

5.3 The Over-Governance Finding

The over-governed result is the central finding. A system with 100% review rate, maximum governance checks, and all high-tier risk gates *looks* perfectly governed by every conventional metric. A threshold-based classifier checking $\text{review_rate} > 0.3 \wedge \text{governance_checks} > 2$ labels it "healthy."

Our structural diagnostic catches the instability because it models *gatedness* — the ratio of passing to total gates. When everything holds, gatedness drops below the health band. The system cannot breathe.

This is not a theoretical curiosity. It corresponds to a recognized organizational failure mode: *process paralysis* — when governance overhead exceeds the team's capacity to comply, and work either stops or routes around the governance through informal channels.

The structural diagnosis makes this formal and testable: a governance configuration where gatedness < 0.2 is structurally unstable, regardless of how many checks are in place.

5.4 Sensitivity Analysis

The well-governed scenario converges for reach values in $[0.1, 0.7]$ — 78% of the tested range. It fails at reach ≥ 0.8 where all gate thresholds are exceeded. The wide convergence region reflects the governance infrastructure (holding gates from checks) providing structural stability.

The sensitivity of each scenario to the reach parameter reveals structural differences. The well-governed configuration's wide convergence region ($[0.1, 0.7]$) reflects its balanced VT distribution: the presence of both low-tier (VT0–VT1) and moderate-tier (VT2) actions creates a mix of passing and holding gates at most reach values. The ungoverned configuration, with all actions at VT0 (threshold $\theta = 0.1$), has all gates passing for any reach above 0.1 — the structural deficit is not in the gates but in the complete absence of coordination bindings.

The over-governed configuration's instability is *reach-invariant*: at any reach below 0.7, the VT3 ($\theta = 0.7$) and VT4 ($\theta = 0.95$) gates hold, producing gatedness well below the health band. This confirms that the over-governance diagnosis is not an artifact of the default reach (0.5) but a robust structural finding.

Table 6: Comparison with Related Approaches

Approach	Monitors	Formal	Structural	Prescribes
Policy engines [14]	✓			
Runtime verif. [11]	✓	✓		
AgentGuard [20]	✓	✓		
MAPE-K [8]	✓			✓
AWARE [5]	✓			✓
LoA/Trust models [16]		✓		
This work	✓	✓	✓	✓

5.5 Baseline Comparison

The structural diagnostic catches the over-governed failure mode that the threshold classifier misses (Figure 5). Table 6 compares our approach with related work along four dimensions.

5.6 Live Demonstration

We provide a self-contained HTML demonstration that simulates a day of governance events in a multi-agent system. As events fire, the governance structure grows, metrics update in real time, and the evaluator shows regulatory actions. The demonstration includes auto-encoded governance events, real-time coordination and gatedness gauges, a regulatory action log with specific remedies, per-agent coordination tracking (trust proxy), and an expression tree preview.

6 Discussion

6.1 What Practitioners Gain

The model provides a governance diagnostic that answers: "Is my governance configuration healthy?" Not "is this action permitted?" (that's policy enforcement) but "is the overall balance of autonomy and oversight working?" With specific remedies when it isn't.

The analogy is a type checker for governance configurations. A type checker does not predict when your program will crash. It tells you whether the structure is sound — and what to fix if it isn't. Similarly, our model does not predict when governance will fail. It tells you whether the governance *configuration* is structurally stable.

Three concrete capabilities follow from this:

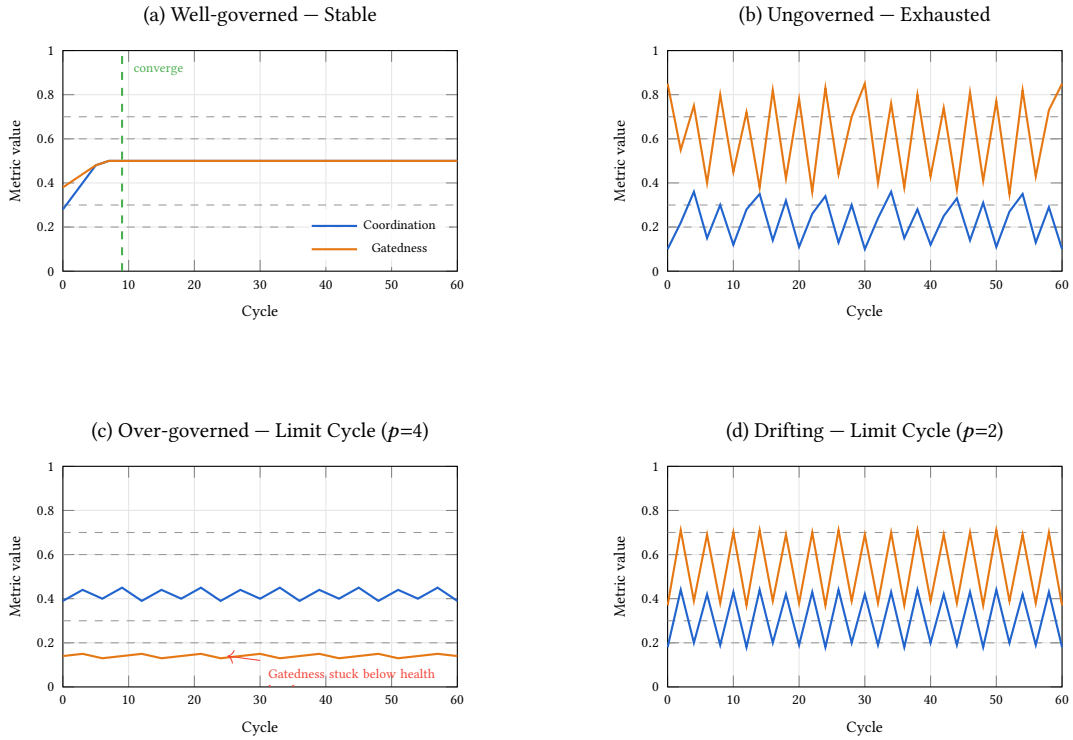

Figure 3: Convergence traces for four governance scenarios. Blue: coordination health. Orange: gatedness. Dashed lines mark health band boundaries (coordination $[0.3, 0.7]$, gatedness $[0.2, 0.6]$). (a) Well-governed converges at cycle 9. (b) Ungoverned oscillates without settling. (c) Over-governed: coordination in band but gatedness stuck at ~ 0.14, below the health band. (d) Drifting: period-2 oscillation crossing band boundaries.

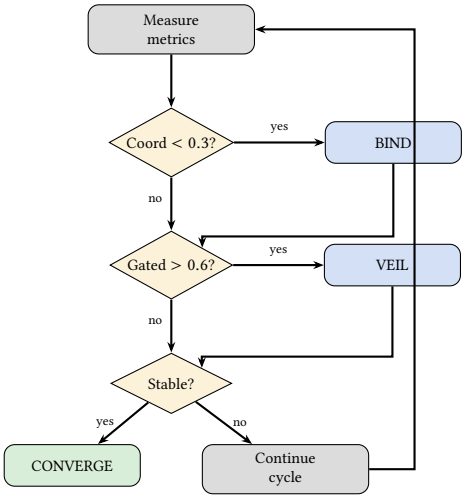

Figure 4: The regulatory cycle. The evaluator measures coordination and gatedness, fires BIND or VEIL when metrics exit the health band, and checks for convergence. The cycle repeats until stable or the step budget is exhausted.

Scenario	Structural diagnostic	Threshold classifier
Well-governed	Stable	Stable
Ungoverned	Exhausted	Unstable
Over-governed	**Limit cycle**	**Stable**
Drifting	Limit cycle	Unstable

Figure 5: Baseline comparison with traffic-light coloring. The over-governed scenario (row 3, highlighted) reveals the key mismatch: the threshold classifier labels it *stable* (green) while the structural diagnostic correctly identifies a *limit cycle* (red).

(1) **Pre-deployment validation.** Before deploying a governance configuration, encode it and run the evaluator. If it converges, deploy. If it enters a limit cycle, diagnose which metric is out of band and adjust the VT distribution or review rate accordingly.
(2) **Continuous monitoring.** During operation, stream governance events into the encoder and track coordination and gatedness over time. The convergence trace (Figure 3) serves as a real-time governance health dashboard.
(3) **Incident forensics.** After a governance failure (missed review, shadow process, unauthorized action), encode the configuration that was active at the time and run the diagnostic. If it

shows a limit cycle or exhaustion, the failure was structural — the configuration was unhealthy regardless of individual compliance.

6.2 Relationship to Existing Tools

Our approach is complementary to runtime policy engines like Microsoft's Agent Governance Toolkit [14]. Policy engines enforce *per-action* compliance ("is this VT2 action approved?"). Our model provides *configuration-level* health monitoring ("is the overall VT distribution stable?"). Both are needed. Enforcement without health monitoring misses structural problems. Health monitoring without enforcement has no teeth.

The relationship to MAPE-K [8] is more nuanced. Our evaluator *is* a MAPE-K loop, but operating on a novel substrate: the expression tree. Traditional MAPE-K monitors observe numerical sensor values (CPU load, response time, queue depth). Our monitor observes the *topology* of the governance structure. This structural substrate gives us algebraic properties (commutativity, associativity, grounding) that numerical monitors lack. The consequence: our regulatory actions (BIND, VEIL) are formally guaranteed to be non-interfering (Property 3) and order-independent (Property 5) — guarantees that are difficult to achieve with numerical control loops where feedback interactions are the norm.

The relationship to runtime verification [11, 20] is complementary but distinct. Runtime verification checks whether an execution trace satisfies a temporal property (φ). Our model checks whether a governance *configuration* occupies a health band. The difference is analogous to the distinction between testing ("does this execution pass?") and type checking ("is this program well-formed?"). Both are valuable; they answer different questions.

6.3 Limitations

Aggregate encoding. The current model encodes a governance configuration's aggregate properties (VT distribution, total handoffs, total checks). It does not capture the *sequence* of events. A configuration that starts governed and drifts ungoverned produces the same encoding as one that drifts then recovers. Trajectory-sensitive encoding is future work.

Sufficient conditions. The convergence conditions (Property 5 of the supplementary) state when the evaluator converges, but we do not characterize all convergent configurations. The sufficient conditions are useful for designing governance configurations guaranteed to be stable.

Synthetic evaluation. The four scenarios are synthetic, though derived from a real system's governance structure. Validation against live sprint telemetry is planned.

Threshold sensitivity. The health band parameters (coordination $[0.3, 0.7]$, gatedness $[0.2, 0.6]$) are configuration choices. Different bands produce different classifications. The over-governance finding is robust (it persists across band widths) but the exact boundary depends on the band.

Single system evaluation. We evaluate on one multi-agent system (CARLOS-OS). While the four scenarios cover the major governance failure modes (under-governance, over-governance, drift), generalizability to other multi-agent architectures (e.g., AutoGen conversation patterns [19], hierarchical agent organizations [3]) requires further study. The algebraic framework is general — any governance system that can express coordination and gating events can be encoded — but the specific health band parameters may need calibration per system.

7 Conclusion

We presented a formal model for governance health monitoring in multi-agent systems. The model auto-encodes governance events into an expression tree, computes coordination health and gatedness from the tree's structure, and classifies the governance configuration as stable, oscillating, or drifting.

The central finding — that over-governance is structurally as unstable as under-governance — formalizes an intuition that practitioners know but cannot currently test. The five proven properties ensure the evaluator behaves predictably, and the comparison with threshold-based classification shows the structural approach catches failure modes that snapshot metrics miss.

Future work. Four directions follow naturally from this work:

(1) Trajectory-sensitive encoding. The current model encodes aggregate governance properties. A trajectory-sensitive variant would encode the *sequence* of events, enabling detection of drift patterns ("governance was healthy until sprint 15, then degraded"). This requires extending the expression tree with temporal operators — a connection to runtime verification [11] that we plan to explore.

(2) Live event stream integration. CARLOS-OS's Redis bus produces governance events in real time. Connecting the encoder to this stream would enable continuous governance health monitoring with sub-second latency, moving from retrospective analysis to live dashboards.

(3) Static governance type system. The current model is a runtime diagnostic. A static type system would determine governance health at *design time* — before deployment. The formal properties (Section 4) provide the foundation: the health band defines the "well-typed" configurations, and the regulatory actions define the type-level rewrites.

(4) Empirical validation. Correlating structural diagnosis with human-perceived governance quality (collected via sprint retrospectives) would validate the model against ground truth. We hypothesize that configurations diagnosed as "limit cycle" correlate with retrospective comments about process friction, and "stable" configurations correlate with smooth sprints.

References

[1] W. Ross Ashby. 1956. *An Introduction to Cybernetics.* Chapman & Hall.
[2] Stafford Beer. 1981. *Brain of the Firm* (2nd ed.). Wiley.
[3] Prithviraj Bose. 2025. From Anarchy to Assembly: A Survey of Governance Frameworks for Collaborative LLM Agent Systems. *International Journal of Engineering Research and Emerging Technology (IJERET)* (2025).
[4] European Parliament. 2024. Regulation (EU) 2024/1689 — The Artificial Intelligence Act. Official Journal of the European Union.
[5] Daniel Feitosa et al. 2025. Breaking the Loop: AWARE is the new MAPE-K. *HAL preprint* (2025).
[6] Kevin Anthony Hoff and Masooda Bashir. 2015. Trust in Automation: Integrating Empirical Evidence on Factors That Influence Trust. *Human Factors* 57, 3 (2015), 407–434.
[7] David Kaber. 2025. Systematic Literature Review of Levels of Automation Taxonomy. *International Journal of Human-Computer Interaction* (2025).

[8] Jeffrey O. Kephart and David M. Chess. 2003. The Vision of Autonomic Computing. *IEEE Computer* 36, 1 (2003), 41–50.

[9] Seonghee Kim et al. 2025. Plan-Then-Execute: User Trust and Team Performance with LLM Agents. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25).*

[10] John D. Lee and Katrina A. See. 2004. Trust in Automation: Designing for Appropriate Reliance. *Human Factors* 46, 1 (2004), 50–80.

[11] Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303.

[12] Hao Li et al. 2025. Governance Architecture for Autonomous Agent Systems: Threats, Framework, and Engineering Practice. *arXiv preprint arXiv:2603.07191* (2025).

[13] Yushuai Li et al. 2024. Generative AI for Self-Adaptive Systems: State of the Art and Research Roadmap. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* (2024).

[14] Microsoft. 2026. Agent Governance Toolkit: Open-source runtime security for AI agents. https://github.com/microsoft/agent-governance-toolkit.

[15] NIST. 2023. *AI Risk Management Framework (AI RMF 1.0).* Technical Report NIST AI 100-1. National Institute of Standards and Technology.

[16] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. 2000. A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans* 30, 3 (2000), 286–297.

[17] Jose Perez Rios. 2025. The Viable System Model and Organizational Pathologies in the Age of AI. *Systems* 13, 9 (2025), 749.

[18] Danny Weyns. 2020. *Introduction to Self-Adaptive Systems: A Contemporary Software Engineering Perspective.* Wiley.

[19] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *Proceedings of the Conference on Language Modeling (COLM).*

[20] Ruijie Xiang et al. 2025. AgentGuard: Runtime Verification of AI Agents. *arXiv preprint arXiv:2509.23864* (2025).

[21] Matthew Zurek et al. 2024. SARI: Shared Autonomy across Repeated Interaction. *ACM Transactions on Human-Robot Interaction* (2024).